CS3460: Data Structures

Lab 07: Graphs

Total Points: 100

Problems

Notes on Grading: Grading will be handled via Gradescope. Any file you submit must begin with the following package statement to be accepted:

```
1 package student;
```

Your code will be compiled and run against a series of tests. You may submit as many times as you would like before the assignment closes. To earn full credit, you must submit to both Gradescope and AsULearn before the assignment deadline.

Submission: Please submit the files `Jugs.java` and `WordLadder.java`.

1. **Water Jugs (50 points):** You managed to get the wolf, the goat, and the cabbage across the river safely, and all that depth-first searching has worked up quite an appetite! It's time for dinner, and you have a head of cabbage to boil.

   You have two water jugs, which have integer sizes $0 < A, B \leq 1000$. In order to boil the cabbage for dinner, you need to measure out exactly C units of water. Write a program named `Jugs.java` that, given A, B, and C, computes whether the task is possible or impossible.

```
Enter A: 3
Enter B: 4
Enter C: 5
Possible: ABCFCFB
```

   Note that the above solution is not optimal, because it performs a few redundant operations (fills the first jug, then fills the second, then empties the first one, wasting a fill and empty action). You may not come up with the same solution as mine, but as long as your solution is *valid*, it is correct.

```
Enter A: 3
Enter B: 6
Enter C: 5
Impossible!
```

   To solve this problem, perform a depth-first search through a graph where your state is represented by a pair of integers (a, b), meaning that Jug 1 contains a units of water and Jug 2 contains b units of water. You will start from the state $(0, 0)$ where both jugs are empty, and any state (a, b) where $a + b = C$ is a valid goal state.

   At any state, there are six possible transitions:

   - `A`: Fill Jug 1 to A
   - `B`: Fill Jug 2 to B
   - `C`: Empty Jug 1 to 0
   - `D`: Empty Jug 2 to 0
   - `E`: Pour Jug 1 into Jug 2 (until 2 is full or 1 is empty)
   - `F`: Pour Jug 2 into Jug 1 (until 1 is full or 2 is empty)

Write a method `solve` that takes three values and returns a `String` with the steps to reproduce if a solution is possible, or `null` if not.

```
1 public static String solve(int a, int b, int c)
2 {
3 }
```

Your solution does not have to be optimal. You can use a two-dimensional array to represent your set of visited nodes. You will need "back pointers" to output your solution.

2. **Word Ladder (50 points):** Consider a word ladder that can be formed between two 4-letter words.

lose → lost → lest → best → beat

Each word is formed by changing a single letter in the previous word. In this problem, we would like to find the shortest path that transitions the starting word to the goal word. All valid words must belong to the provided dictionary (file `dictionary4.txt`) and are made up of lowercase letters in the range a-z.

The solution is to perform a breadth-first search through the implicit graph, where each node corresponds to a word, and edges are formed by words that share all but one letter in common. To solve this problem effectively, we will need two data structures, a `HashMap` and a `Queue` (for this problem, you may use the classes provided in the Java Collections library).

The `HashMap` will be used for two things: marking words as "visited", and storing back pointers. Both of which can be done with one `HashMap` by associating each entry in the dictionary with an empty string (or null) when it hasn't yet been visited, and updating that value to the predecessor for the breadth-first search when it is visited. Therefore, if a key is matched with a non-empty value, it has been visited.

The `Queue` can be used for a standard FIFO work queue. Keep in mind that `Queue` in Java is an interface that is implemented by `LinkedList`, among other classes.

Write a program named `WordLadder.java` that loads all of the words from the provided dictionary into a `HashMap` (associated with an empty string or null), asks the user for a start and end word, and then prints whether it is possible, and if so, the shortest sequence between the two words.

```
Enter the start word: lose
Enter the end word: beat
Possible!
lose
lost
lest
best
beat
```

```
Enter the start word: quiz
Enter the end word: exam
Impossible!
```

Write a method `solve` that returns a `List<String>` with the word ladder, or `null` if no such word ladder exists.

```
1 public static List<String> solve(String dictfile, String start, String end)
2 {
3 }
```